Solucionario Java — Actividad 2: Principios SOLID

Curso: Programación Orientada a Objetos II — Semana 02

Este documento resuelve "Actividad-2-POOII-Resolución de Ejercicios": identificar las violaciones a los principios SOLID en el sistema de gestión de empleados dado, y refactorizar el código aplicando cada principio.

Índice

1. Violaciones identificadas en el código original
2. Clase Empleado.java (abstracta)
3. Clases Gerente / Desarrollador / Practicante
4. Interfaces RepositorioEmpleado y GeneradorReporte
5. Clase SistemaGestionEmpleados.java
6. Clase Main.java

1. Violaciones identificadas en el código original

SRP — Principio de responsabilidad única

Empleado mezclaba cuatro responsabilidades distintas: guardar sus propios datos, calcular el pago, persistir en base de datos (guardarEnBaseDeDatos) y generar reportes (generarReporte). Un cambio en la forma de persistir o reportar obliga a modificar la misma clase que representa al empleado.

OCP — Abierto/cerrado

calcularPago() decide el cálculo con una cadena de if/else sobre el String tipo. Agregar un nuevo tipo de empleado (por ejemplo "Gerente Senior") exige modificar ese método existente, en lugar de simplemente extender el sistema.

LSP — Sustitución de Liskov

Al no existir jerarquía de clases por tipo de empleado (todo es la misma clase Empleado con un atributo String tipo), no hay una verdadera relación de sustitución; el diseño no permite tratar distintos tipos de empleado de forma polimórfica y seguirlo haciendo de forma segura al agregar nuevos tipos.

ISP — Segregación de interfaces

No existen interfaces: la persistencia y el reporte están "pegados" a Empleado, forzando a cualquier consumidor de la clase a cargar con métodos (guardarEnBaseDeDatos, generarReporte) que no siempre necesita.

DIP — Inversión de dependencias

SistemaGestionEmpleados depende directamente de la clase concreta Empleado y de sus métodos de persistencia/reporte embebidos, en vez de depender de abstracciones; esto impide, por ejemplo, cambiar el mecanismo de guardado sin tocar SistemaGestionEmpleados.

`Solución general`: se separa Empleado en una jerarquía polimórfica (Gerente, Desarrollador, Practicante) que resuelve SRP/OCP/LSP en el cálculo del pago, y se extraen la persistencia y el reporte a interfaces (RepositorioEmpleado, GeneradorReporte) inyectadas en SistemaGestionEmpleados, resolviendo ISP y DIP.

Volver al índice · Siguiente: 2. Clase Empleado.java (abstracta)

2. Clase Empleado.java (abstracta)

Ahora es abstracta y sólo conserva los datos comunes (nombre, salario) y el contrato calcularPago(), que cada tipo de empleado implementa a su manera (resuelve OCP: agregar un tipo nuevo = una clase nueva, sin tocar esta).

`src/main/java/poo2/s02/solid/Empleado.java`

```
package poo2.s02.solid;

public abstract class Empleado {

    private String nombre;
    private double salario;

    public Empleado(String nombre, double salario) {
        this.nombre = nombre;
        this.salario = salario;
    }

    public String getNombre() {
        return nombre;
    }

    public double getSalario() {
        return salario;
    }

    public abstract double calcularPago();
}
```

Volver al índice · Siguiente: 3. Clases Gerente / Desarrollador / Practicante

3. Clases Gerente / Desarrollador / Practicante

Cada subclase implementa calcularPago() con la regla que antes vivía en el if/else.
Cualquiera puede sustituir a Empleado sin romper el comportamiento del sistema (LSP).

src/main/java/poo2/s02/solid/Gerente.java

```
package poo2.s02.solid;

public class Gerente extends Empleado {

    private static final double BONO_GERENTE = 1000;

    public Gerente(String nombre, double salario) {
        super(nombre, salario);
    }

    @Override
    public double calcularPago() {
        return getSalario() + BONO_GERENTE;
    }
}
```

src/main/java/poo2/s02/solid/Desarrollador.java

```
package poo2.s02.solid;

public class Desarrollador extends Empleado {

    public Desarrollador(String nombre, double salario) {
        super(nombre, salario);
    }

    @Override
    public double calcularPago() {
        return getSalario();
    }
}
```

src/main/java/poo2/s02/solid/Practicante.java

```
package poo2.s02.solid;

public class Practicante extends Empleado {

    private static final double FACTOR_PRACTICANTE = 0.5;

    public Practicante(String nombre, double salario) {
        super(nombre, salario);
```

```
    }

    @Override
    public double calcularPago() {
        return getSalario() * FACTOR_PRACTICANTE;
    }
}
```

Volver al índice · Siguiente: 4. Interfaces RepositorioEmpleado y GeneradorReporte

4. Interfaces RepositorioEmpleado y GeneradorReporte

Se separan la persistencia y el reporte en interfaces pequeñas y específicas (ISP), cada una con una única implementación concreta que reproduce el comportamiento original (imprimir por consola).

`src/main/java/poo2/s02/solid/RepositorioEmpleado.java`

```
package poo2.s02.solid;

public interface RepositorioEmpleado {
    void guardar(Empleado empleado);
}
```

`src/main/java/poo2/s02/solid/RepositorioEmpleadoBaseDeDatos.java`

```
package poo2.s02.solid;

public class RepositorioEmpleadoBaseDeDatos implements RepositorioEmpleado {

    @Override
    public void guardar(Empleado empleado) {
        System.out.println("Guardando empleado " + empleado.getNombre() + " en la
base de datos...");
    }
}
```

`src/main/java/poo2/s02/solid/GeneradorReporte.java`

```
package poo2.s02.solid;

public interface GeneradorReporte {
    void generar(Empleado empleado);
}
```

`src/main/java/poo2/s02/solid/GeneradorReporteConsola.java`

```
package poo2.s02.solid;

public class GeneradorReporteConsola implements GeneradorReporte {

    @Override
    public void generar(Empleado empleado) {
        System.out.println("Generando reporte para el empleado " +
empleado.getNombre() + "...");
    }
}
```

Volver al índice · Siguiente: 5. Clase SistemaGestionEmpleados.java

5. Clase SistemaGestionEmpleados.java

Ahora depende de las abstracciones RepositorioEmpleado y GeneradorReporte, recibidas por constructor (inyección de dependencias), en vez de conocer los detalles de guardado o reporte del Empleado — resuelve DIP.

`src/main/java/poo2/s02/solid/SistemaGestionEmpleados.java`

```
package poo2.s02.solid;

public class SistemaGestionEmpleados {

    private final RepositorioEmpleado repositorio;
    private final GeneradorReporte generadorReporte;

    public SistemaGestionEmpleados(RepositorioEmpleado repositorio, GeneradorReporte
generadorReporte) {
        this.repositorio = repositorio;
        this.generadorReporte = generadorReporte;
    }

    public void procesarEmpleado(Empleado empleado) {
        double pago = empleado.calcularPago();
        System.out.println("Pago calculado: " + pago);
        repositorio.guardar(empleado);
        generadorReporte.generar(empleado);
    }
}
```

Volver al índice · Siguiente: 6. Clase Main.java

6. Clase Main.java

Reproduce exactamente el escenario del enunciado (un gerente, un desarrollador y un practicante), ahora usando la jerarquía polimórfica y las dependencias inyectadas.

`src/main/java/poo2/s02/solid/Main.java`

```
package poo2.s02.solid;

public class Main {

    public static void main(String[] args) {
        Empleado gerente = new Gerente("Juan", 5000);
        Empleado desarrollador = new Desarrollador("Ana", 3000);
        Empleado practicante = new Practicante("Luis", 1000);

        RepositorioEmpleado repositorio = new RepositorioEmpleadoBaseDeDatos();
        GeneradorReporte generadorReporte = new GeneradorReporteConsola();
        SistemaGestionEmpleados sistema = new SistemaGestionEmpleados(repositorio,
generadorReporte);

        sistema.procesarEmpleado(gerente);
        sistema.procesarEmpleado(desarrollador);
        sistema.procesarEmpleado(practicante);
    }
}
```

Volver al índice